

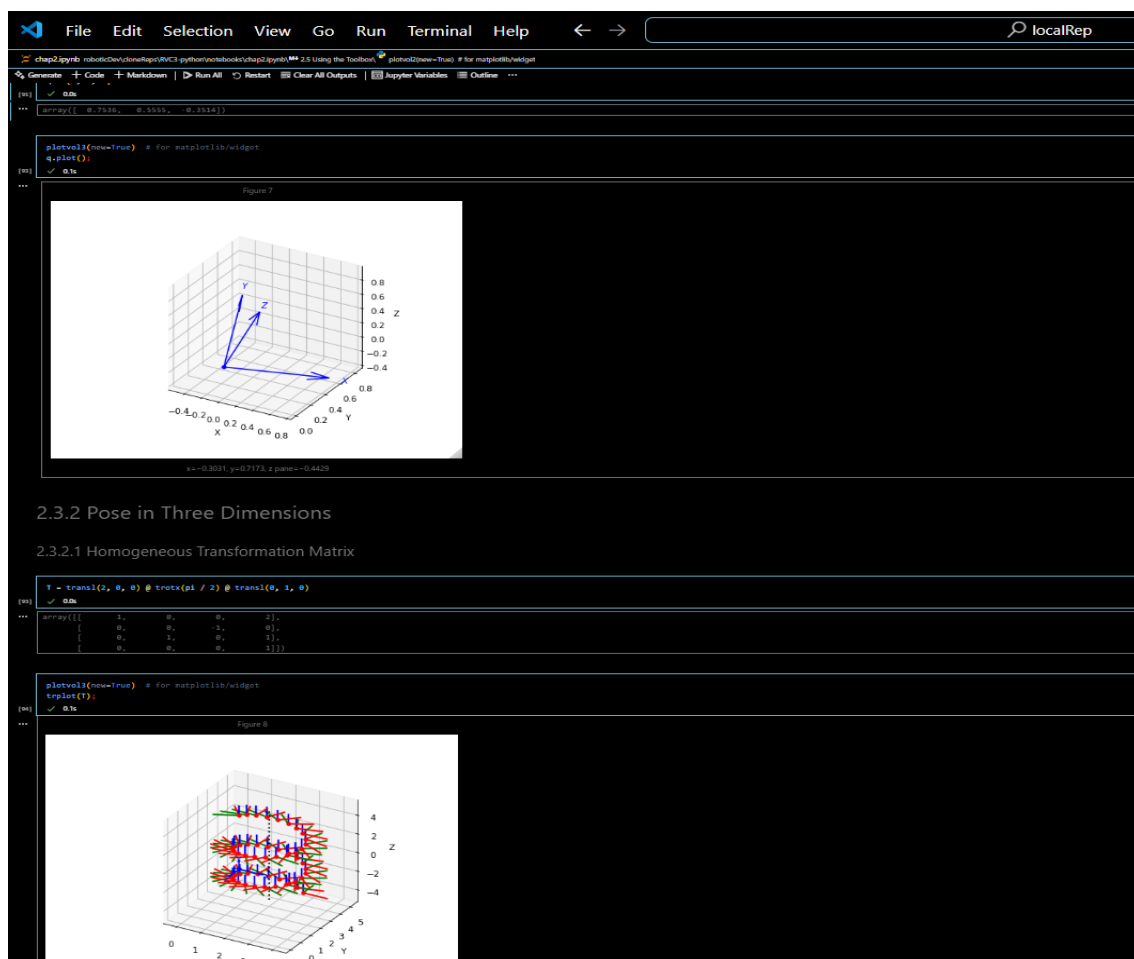
GIT CLONING REPS:

```
[ (< | D | 1 | r | r | r | r | cloneReps) (<?main =>) <3ms> ]
[ >>>>> git clone git@github.com:petercorke/robotics-toolbox-python.git8:17 ]
Cloning into 'robotics-toolbox-python' ...
remote: Enumerating objects: 27011, done.
remote: Counting objects: 100% (724/724), done.
remote: Compressing objects: 100% (106/106), done.
remote: Total 27011 (delta 663), reused 618 (delta 618), pack-reused 26287 (from 2)
Receiving objects: 100% (27011/27011), 174.11 MiB | 12.13 MiB/s, done.
Resolving deltas: 100% (19130/19130), done.
Updating files: 100% (1980/1980), done.
```

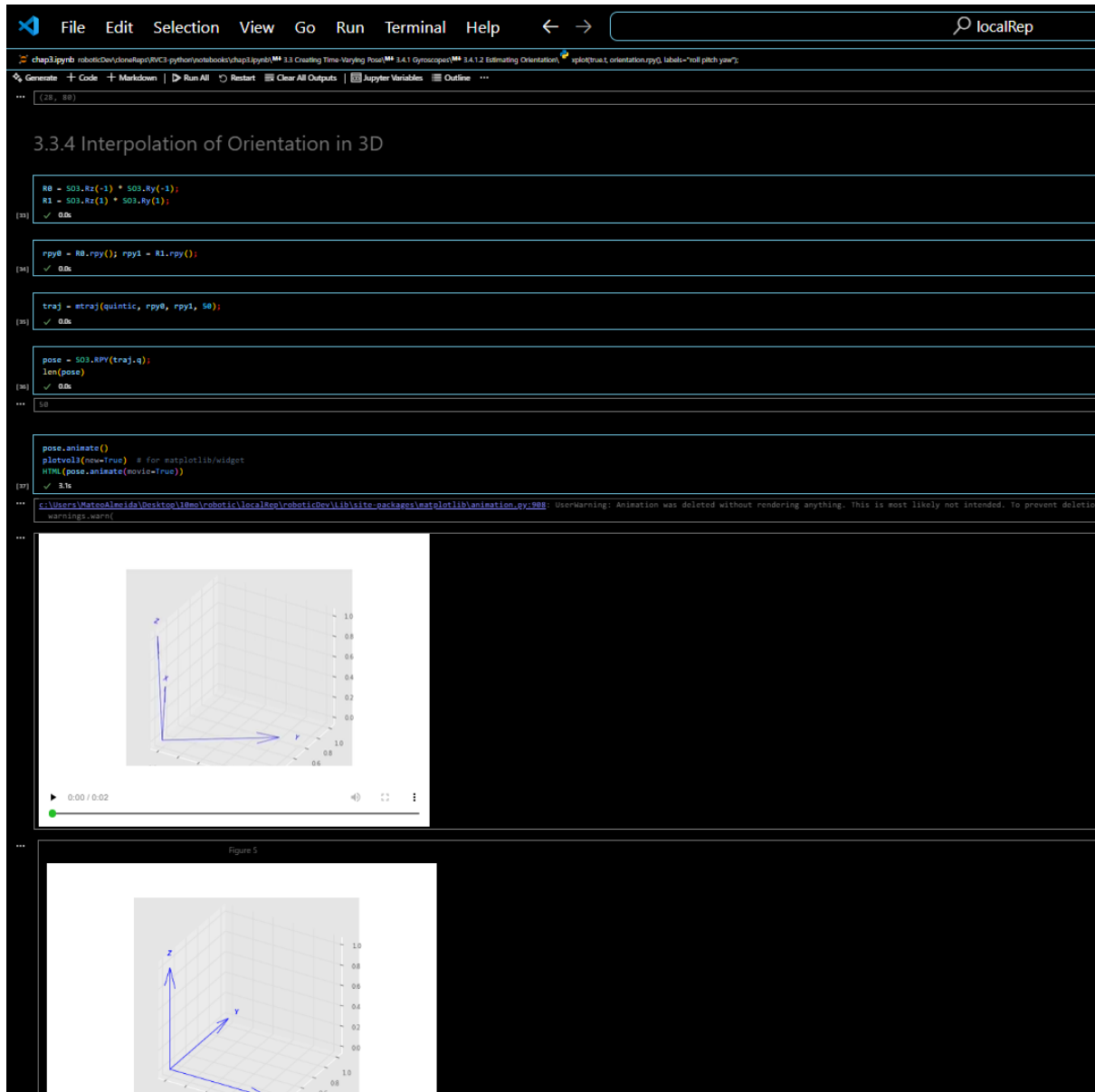
```
[ (< | D | 1 | r | r | r | r | cloneReps) (<?main =>) <1.778s> ]
[ >>>>> git clone git@github.com:petercorke/RVC3-python.git <22:00:15> ]
Cloning into 'RVC3-python' ...
remote: Enumerating objects: 2678, done.
remote: Counting objects: 100% (25/25), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 2678 (delta 17), reused 13 (delta 12), pack-reused 2653 (from 2)
Receiving objects: 100% (2678/2678), 83.44 MiB | 4.31 MiB/s, done.
Resolving deltas: 100% (1554/1554), done.
Updating files: 100% (717/717), done.
```

proof of using **on LOCAL REP** (notebooks copied from this LOCAL DIRECTORY)

On chap2.ipynb



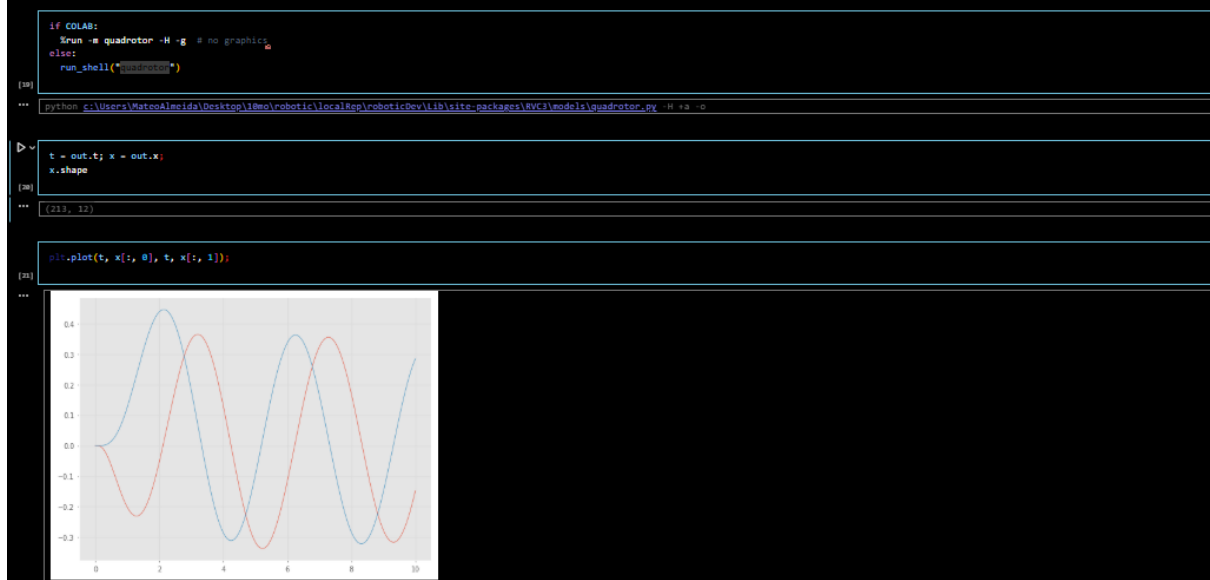
On chap3.ipynb



On AERIAL ROBOTS from CHAP4.ipynb



4.2 Aerial Robots



4.4 Wrapping Up

And so...

File Edit Selection View Go Run Terminal Help

chap6.ipynb roboticDev\localRep\JVC3-python\notebooks\chap6.ipynb 5.1 Introduction to Reactive Navigation 5.1.2 Simple Automata

Generate + Code + Markdown | ▶ Run All ⏹ Restart 🗑 Clear All Outputs | 📄 Jupyter Variables 📄 Outline ...

```

# helper function to run bdsim in a subprocess and transfer results using a pickle file
import pickle
def run_shell(tool, **params):
    global out
    pyfile = os.path.join(os.path_...[0], "models", tool+".py")
    cmd = "python (pyfile) -H -a -o"
    for key, value in params.items():
        cmd += f' --global "{key}-{value}"'
    print(cmd)
    os.system(cmd)
    with open("bd.out", "rb") as f:
        out = pickle.load(f)

# ----- standard imports ----- #
import numpy as np
import os
from math import pi
%.set_printoptions(
    linewidth=120, formatter={
        'float': lambda x: f"{0:8.4g}" if abs(x) < 1e-10 else f"{x:8.4g}"})
np.random.seed(0)
from spatialmath import *
from spatialmath.base import *
from roboticstoolbox import *

```

(3)

5.1 Introduction to Reactive Navigation

5.1.1 Braitenberg Vehicles

```

def sensorfield(x, y):
    xc, yc = (50, 50)
    return 200 / ((x - xc) ** 2 + (y - yc) ** 2 + 200)

```

(3)

NOTE

There are issues in running bdsim from inside Jupyter. We have two options, disable all graphics (which is a bit dull) or run bdsim from a subprocess with its own popout windows.

For Colab we can only do the first option.

Otherwise, we use the wrapper function `run_shell()` defined in the first cell to spawn a new Python instance to run the block diagram which will pop up new windows on your screen to display a simple animation showing the v

- parameter values are passed to bdsim as `--global` command line options.
- the `-o` command line option to `bdsim` causes it to write results as a pickle file, which are then imported back into Jupyter.

```

if COLAB:
    %run -m braitenberg -H -g # no graphics
else:
    run_shell("braitenberg")

```

(3)

```
python g:\Users\MateoAlmeida\Desktop\l0m\robotic\localRep\roboticDev\lib\site_packages\JVC3\models\braitenberg.py -H -a -o
```

```
plt.plot(out.x[:,0], out.x[:,1]);
```

(4)

Could not render content for application/vnd.jupyter.widget-view+json
[{"model_id":"35b01eaf974fca54b95efb16ee419","version_major":2,"version_minor":0}]

5.1.2 Simple Automata

▶ 9 cells hidden

5.2 Introduction to Map-Based Navigation

5.3 Planning with a Graph-Based Map

Chap6.ipynb...

File Edit Selection View Go Run Terminal Help

chapl.ipynb inbetsDev\done\apps\JVC3-python\notebooks\chapl.ipynb
Robotic, Vision & Control for Python Chapter 6: Localization and Mapping
There are some minor code changes compared to the book. These are to support

Generate Code Markdown Run All Restart Clear All Outputs Jupyter Variables Outline
11 cells hidden

6.3 Creating a Landmark Map
16 cells hidden

6.4 Simultaneous Localization and Mapping
5 cells hidden

6.5 Pose-Graph SLAM

NOTE: Minor changes to ensure Jupyter pretty printing of SymPy equations. Introduced underscore to indicate subscripting, and changed t to theta which is printed as θ .

```

import sympy
xi, yi, ti, xj, yj, tj = sympy.symbols("x_i y_i theta_i x_j y_j theta_j")
xm, ym, tm = sympy.symbols("x_m y_m theta_m")
xi_e = SE2(xm, ym, tm).inv() * SE2(xi, yi, ti).inv() \
    * SE2(xj, yj, tj)
fk = sympy.Matrix(sympy.simplify(xi_e.xyt()))

```

```

A1 = sympy.simplify(fk.jacobian([xi, yi, ti]))
A1.shape

```

```

pg = PoseGraph("data/pg1.g2o");

```

```

pg.plot();

```

Last one was chap7.ipynb from arm kinematics.

File

Edit

Selection

View

Go

Run

Terminal

Help

←

→

localRep

chap7.ipynbroboticsDev/donellapp/JVC3-pyhton/notebooks/chap7.ipynb7.1 Forward Kinematics7.1.1 Forward kinematics from a pose graph7.1.1.1 2-Dimensional (Planar) Robotic Arms# e.teach() # not working in Jupyter

Generate

Code

Markdown

Interrupt

Restart

Clear All Outputs

Go To

Jupyter Variables

Outline

Copyright (c) 2021 - Peter Corke

1 cell hidden

7.1 Forward Kinematics

7.1.1 Forward kinematics from a pose graph

7.1.1.1 2-Dimensional (Planar) Robotic Arms

31 = 1

e = SE2.R() * SE2.Tx(a1)

0.0s

len(e)

0.0s

2

e

0.0s

R(q)Tx(1)

e.fline(p1 / 6)

0.0s

0.866-0.50.8660.501

SE2.Rot(p1 / 6) * SE2.Tx(a1)

0.0s

0.866-0.50.8660.501

e.teach(0) # not working in Jupyter

e.plot(0)

2.1s

Figure 1

